

Введение в Android



Kotlin

Kotlin - статически типизированный, объектно-ориентированный язык программирования, работающий поверх Java Virtual Machine.

Полностью совместим с Java.

Kotlin. Свойства

Ненуллабельные свойства

Kotlin

```
val immutable: Int = 5
var mutable: Int = 5
```

Нуллабельные свойства

Kotlin

```
var propertyNullable: Int? = null
var property: Int = null // Error
propertyNullable.toDouble() // Error
propertyNullable!!.toDouble() // NullPointerException
propertyNullable?.toDouble() ?: 0.0
propertyNullable?.toDouble() // Null if propertyNullable == null
```

Автовыведение типа

Kotlin

```
val immutable = 5 // Int
```



Kotlin. Области видимости

При объявлении на самом высоком уровне:

- **public** - модификатор доступа по умолчанию. Весь код данного объявления будет виден отовсюду;
- **private** - видимость только внутри файла, где было объявлено;
- **internal** - видимость будет распространяться на весь модуль;
- **protected** запрещено использовать в объявлениях “высокого уровня”.

Kotlin. Области видимости

При объявлении членов класса:

- **public** - любой клиент, который видит объявленный класс, видит его **public** члены;
- **private** - видимость только внутри класса, включая его члены;
- **protected** - то же самое, что **private** + видимость в наследниках.
- **internal** - любой клиент внутри модуля, который видит объявленный класс, видит и его **internal** члены;

Если вы переопределите **protected** или **internal** член и явно не укажете его видимость, переопределённый элемент будет иметь тот же модификатор, что и исходный.

Kotlin. Свойства

СВОЙСТВА

Kotlin

```
var name: String = ""
    get() = if (field.isBlank()) {
        "Nameless"
    } else {
        "My name is $field"
    }
    set(value) {
        field = value.uppercase()
    }
fun main() {
    println(name) // Nameless
    name = "john wick"
    println(name) // My name is JOHN WICK
}
```

Kotlin. Основные типы

Основные типы

Kotlin

```
val byte: Byte = 127
val short: Short = 32767
val int: Int = 2_147_483_647
val long: Long = 10L
val float: Float = 10.5f
val double: Double = 10.5
val string: String = "Some String"
val char: Char = 'c'
val boolean: Boolean = true // false
```

Kotlin. Основные коллекции

Основные коллекции

Kotlin

```
val list = listOf(1, 2, 4, 30) // 1, 2, 4, 30
val map = mapOf(1 to "one", 2 to "two") // {1=one, 2=two}
val set = setOf("one", "two", "one", "two") // two, one
val linkedList = LinkedList(list) // 1, 2, 4, 30
```

Чтобы получить доступ к мутабельным имплементациям **list**, **map**, **set**, нужно использовать коллекции с приставкой **mutable** (*mutableListOf*, ...)

Kotlin. If else

Условные выражения

Kotlin

```
var max: Int
if (a < b) max = b
// With else
if (a > b) {
    max = a
} else {
    max = b
}
// As expression
max = if (a > b) a else b
```

Kotlin. When

Условные выражения

Kotlin

```
when (x) {  
    1 -> print("x == 1")  
    2 -> print("x == 2")  
    else -> print("x is $x")  
}  
  
when {  
    x == 1 -> print("x == 1")  
    x > 2 -> print("x > 2")  
    else -> print("x is $x")  
}  
  
when(x) {  
    1, 2 -> print("x == 0 or x == 1")  
    else -> print("x is $x")  
}
```

Kotlin. When

Условные выражения

Kotlin

[illegible]

Kotlin. For

Цикл For

Kotlin

```
val colors = listOf(Color.BLACK, Color.WHITE)
for (color in colors) print(color)
// Итерация по массиву через индексы
for (i in colors.indices) print(colors[i])
// Пример выше - аналог следующей записи:
for (i in 0 until colors.size) print(colors[i])
// Может быть блоком
for (color: Color in colors) {
    print(color)
}
for (i in 0..10) ... // от 0 до 10 включительно, шаг 1
for (i in 0 until 10) ... // от 0 до 9, шаг 1
for (i in 0 until 10 step 2) ... // от 0 до 9, шаг 2
for (i in 10 downTo 0 step 2) ... // от 10 до 0, шаг 2
```

Kotlin. While, continue, break

Цикл While

Kotlin

```
while (x > 0) {  
    x--  
}  
do {  
    val x = (0..10).random()  
} while (x != 0)  
  
fun doSomethingRandom(): Int {  
    do {  
        val x = (0..10).random()  
        if (x < 8) continue  
        if (x == 9) return (0..x).random()  
        if (x == 10) break  
    } while (x != 0)  
    return -1  
}
```

Kotlin. Функции

functions

Kotlin

```
fun someFun() {  
    // some code  
}  
fun someOperation(  
    operand1: Int,  
    operand2: Int,  
): Int =  
    operand1 + operand2
```

functions

Kotlin

```
fun someFun(): Unit {  
    // some code  
}  
  
fun someOperation(  
    operand1: Int,  
    operand2: Int,  
): Int {  
    return operand1 + operand2
```

functions

Kotlin

```
val result = someOperation(operand1 = 10, operand2 = 20)
```

Kotlin. Функциональный тип

functional type

Kotlin

```
val funType: () -> Unit = {}  
fun someOperation(  
    action1: (Int) -> String,  
    action2: (String) -> Unit,  
) {  
    action2(  
        action1(1)  
    )  
}  
someOperation(  
    action1 = { it.toString() },  
    action2 = { action1Res -> println(action1Res) } //  
    "1"  
)  
someOperation(  
    action1 = Int::toString,  
    action2 = ::println, // "1"  
)
```

Kotlin. Any

В Kotlin все неявно наследуется от универсального класса **Any** (аналог Object в Java).

Any type

Kotlin

```
public open class Any public constructor() {  
    public open operator fun equals(other:Any?): Boolean  
    {}  
    public open fun hashCode(): Int {}  
    public open fun toString(): String {}  
}
```

Kotlin. Классы

Classes

Kotlin

```
class Spider(  
    val eyes: Int,  
    val legs: Int,  
) {  
    constructor() : this(eyes = 8, legs = 8) {  
        println("Secondary constr: color $color")  
    }  
    var color = "Black"  
    init {  
        println("Primary constr: eyes = $eyes, legs = $legs")  
    }  
}  
  
val spider1 = Spider()  
val spider2 = Spider(eyes = 6, legs = 6)
```

Вывод:

```
Primary constr: eyes = 8, legs = 8  
Secondary constr: color Black  
Primary constr: eyes = 6, legs = 6
```

Kotlin. Классы

Classes

Kotlin

```
abstract class Insect(  
    val eyes: Int,  
    val legs: Int,  
) {  
    open val color: String = ""  
    abstract fun getDescription(): String  
}  
  
class Spider(  
    eyes: Int,  
    legs: Int,  
) : Insect(eyes, legs) {  
    override val color: String = "Black"  
    override fun getDescription(): String =  
        "This is a spider"  
}
```

По умолчанию все классы, их свойства и функции имеют модификатор:

- **final** - не могут быть унаследованы или переопределены.
- **open** - позволяет унаследоваться от класса или, если применено к свойству или функции, позволяет переопределить его, при желании.
- **abstract** - делает класс абстрактным. Такой класс нельзя создать напрямую. Модификатор так же применим к свойствам и функциям. (Такие свойства и функции не будут иметь базовой реализации)

Kotlin. Companion object

Companion object

Kotlin

```
class MathClass {  
    companion object {  
        const val G_EARTH = 9.8  
        fun gravityOnEarth(m: Double) = m * G_EARTH  
    }  
}  
  
fun main() {  
    println(  
        MathClass.gravityOnEarth(m = 10.0)  
    )  
}
```

Еще один способ обратиться к **companion object**:

Companion object

Kotlin

```
MathClass.Companion.gravityOnEarth(m = 10.0)
```

Kotlin. Data Class

Data class

Kotlin

```
data class Point(  
    val x: Double,  
    val y: Double,  
)  
fun main() {  
    val point1 = Point(x = 5.0, y = 5.0)  
    val point2 = Point(x = 5.0, y = 5.0)  
    val (x, y) = point1  
  
    println("(x = $x; y = $y)")  
    println(point1)  
    println(point1 == point2)  
}
```

Вывод:

```
(x = 5.0; y = 5.0)  
PointData(x=5.0, y=5.0)  
true
```

Kotlin. Data Class

Data class

Kotlin

```
class Point(  
    val x: Double,  
    val y: Double,  
)  
fun main() {  
    val point1 = Point(x = 5.0, y = 5.0)  
    val point2 = Point(x = 5.0, y = 5.0)  
    // val (x, y) = point1 - Error  
    println(point1)  
    println(point1 == point2)  
}
```

Вывод:

Point@6659c656

false

Kotlin. Data Class

Компилятор автоматически генерирует методы (На основе параметров главного конструктора):

1. **equals** - сравнение объектов на равенство
2. **hashCode** - хэш-код
3. **toString** - строковое представление класса
4. **componentN** - деструктурирующее объявление
5. **copy** - копирует класс вместе с данными

Требования к **data** классу:

1. Главный конструктор должен содержать хотя бы одно свойство;
2. Все свойства главного конструктора должны быть **val** или **var**;
3. Data классы не могут быть **abstract**, **open**, **sealed** и **inner**.

Data class

Kotlin

```
val (x, y) = point1  
point1.copy(x = 10.0)
```

Kotlin. Objects

Singleton

Kotlin

```
object Constants {  
    const val SOME_VALUE = "some value"  
    fun getSomeString() = "some string"  
}  
fun main() {  
    Constants.SOME_VALUE  
    Constants.getSomeString()  
    println(Constants)  
}
```

Вывод:

Constants@6ff3c5b5

Kotlin. Data Objects

toString - возвращает имя объекта;

equals - сравнивает имя объекта;

hashCode - возвращает одно число для всех экземпляров объекта (при рефлексии).

Singleton

Kotlin

```
data object Constants {  
    const val SOME_VALUE = "some value"  
    fun getSomeString() = "some string"  
}  
fun main() {  
    println(Constants)  
}
```

Вывод:
Constants

Kotlin. Анонимные объекты

Анонимный объект без наследования:

Анонимный объект

Kotlin

```
val helloWorld = object {  
    val hello = "Hello"  
    val world = "World"  
    override fun toString() = "$hello $world"  
}  
print(helloWorld)
```

Анонимный объект с наследованием:

Анонимный объект

Kotlin

```
window.addMouseListener(  
    object : MouseAdapter() {  
        override fun mouseClicked(e: MouseEvent) { /*...*/ }  
        override fun mouseEntered(e: MouseEvent) { /*...*/ }  
    }  
)
```

Вывод: Hello World

Kotlin. Enum

Enum class

Kotlin

```
enum class Color {  
    RED, GREEN, BLUE  
}  
  
enum class Color(val rgb: Int) {  
    RED(0xFF0000),  
    GREEN(0x00FF00),  
    BLUE(0x0000FF),  
    UNKNOWN(-1) {  
        override fun nameWithRgb(): String = "UNKNOWN"  
    };  
    open fun nameWithRgb() = "$this rgb = $rgb"  
}  
  
fun main() {  
    println(Color.RED.nameWithRgb())  
    println(Color.UNKNOWN.nameWithRgb())  
}
```

Вывод:

RED rgb = 16711680

UNKNOWN

Kotlin. Интерфейсы

В отличие от классов, множественное наследование интерфейсов разрешено

Interfaces

Kotlin

```
interface SomeInterface {  
  
    val prop: Int // abstract  
    val propertyWithImplementation: String  
        get() = "property"  
    fun foo() // abstract  
    fun bar() {  
        println("SomeInterface bar()")  
    }  
}  
  
class Child : SomeInterface {  
  
    override val prop: Int = 0  
    override fun foo() {  
        println("Child foo()")  
    }  
}
```

Kotlin. Sealed

Sealed классы и интерфейсы обеспечивают ограниченную область наследования. Таким образом, компилятор знает обо всех реализациях во время компиляции. Область ограничена пакетом, в котором объявлен sealed класс, наследование за пределами пакета - запрещено.

Преимущества:

1. Удобно использовать в месте с when, иерархия классов известна во время компиляции;
2. В отличие от Enum, можно создать несколько экземпляров наследника sealed класса;
3. Гарантия использования вашего API по назначению.

Kotlin. Sealed

Sealed class

Kotlin

```
sealed class State {  
    open val isLoading: Boolean = false  
    data object Loading : State() {  
        override val isLoading: Boolean = true  
    }  
    data class Content(  
        val data: Any,  
    ) : State()  
    data class Error(  
        val cause: Exception,  
    ) : State()  
}
```

Kotlin. Sealed

Sealed class

Kotlin

```
observeState().onEach { state ->
    when (state) {
        is State.Loading -> showLoading()
        is State.Content -> drawContent(state.data)
        is State.Error -> sendError(state.cause)
    }
}

fun observeState(): Flow<State> = flowOf(
    State.Loading,
    State.Content(data = Any()),
    State.Error(cause = IllegalStateException("")),
)
```

Kotlin. Функции расширения

Extension functions

Kotlin

```
fun Color.isExists(): Boolean =
    this != Color.UNKNOWN
fun main() {
    println(Color.RED.isExists())
    println(Color.UNKNOWN.isExists())
}
```

По своей сути, функция расширения - синтаксический сахар, которая представляется в виде:

Extension functions

Kotlin

```
fun isExists(color: Color): Boolean =
    color != Color.UNKNOWN
```

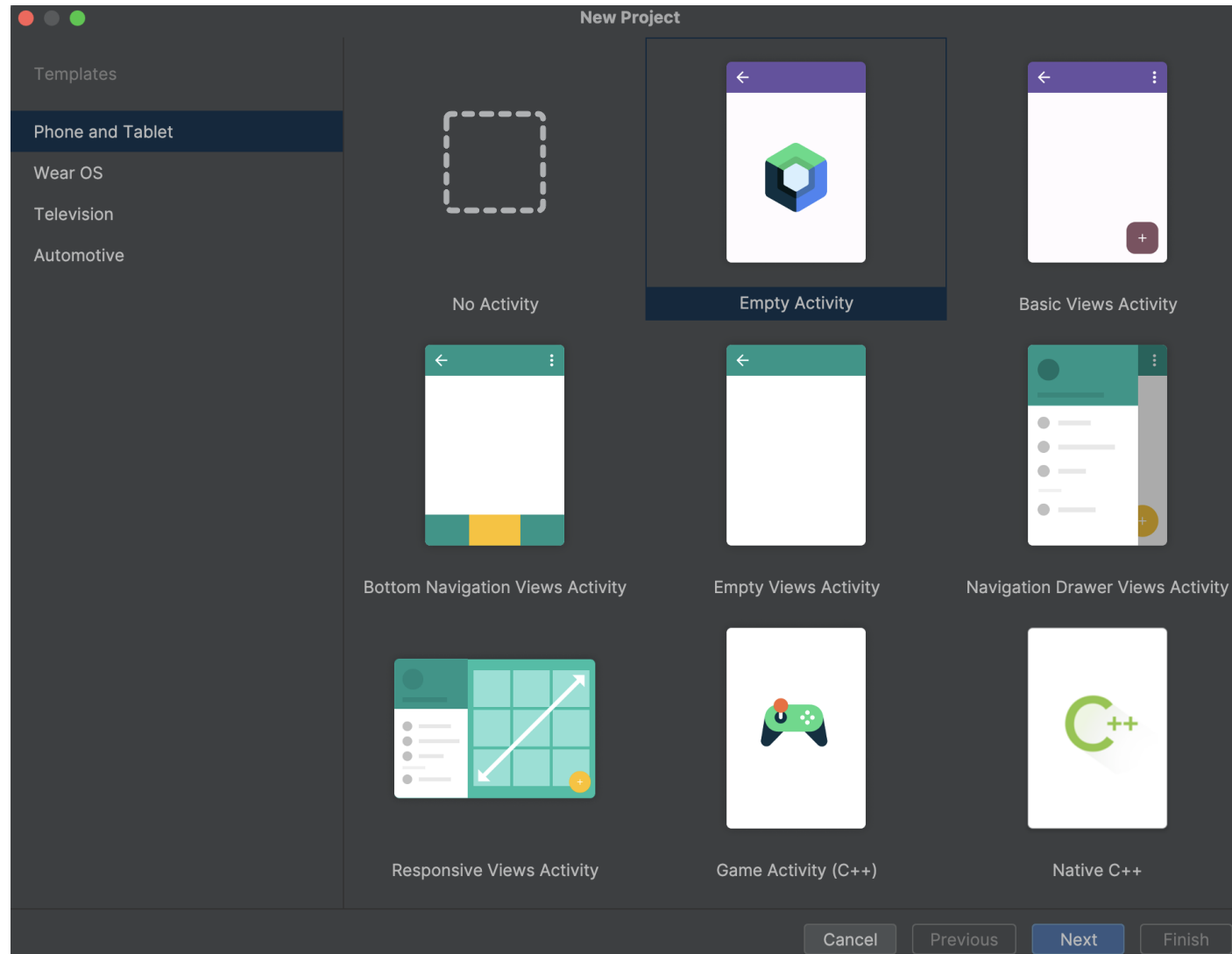
Kotlin. Вспомогательные функции

Functions with receiver

Kotlin

```
class Person(var name: String = "", var age: Int = 0)
class Driver(var name: String = "", var age: Int = 0)
val person = Person().apply { name = "Bob" }
//apply - объект внутри лямбды доступен через this, по завершению возвращается this объект
val person = Person().also { log(it.name) }
//also - объект внутри лямбды доступен через it, по завершению возвращается this объект
val driver: Driver = person?.let { convertToDriver(it) } ?: Driver()
//let - объект внутри лямбды доступен через it, по завершению возвращается объект из return
val driver: Driver = person?.run { convertToDriver(this) } ?: Driver()
//run - объект внутри лямбды доступен через this, по завершению возвращается объект из return
with(person) {
    println("$name $age")
}
//with - объект внутри лямбды доступен через this, по завершению возвращается объект из return
```

Android. Проект



Android. Проект

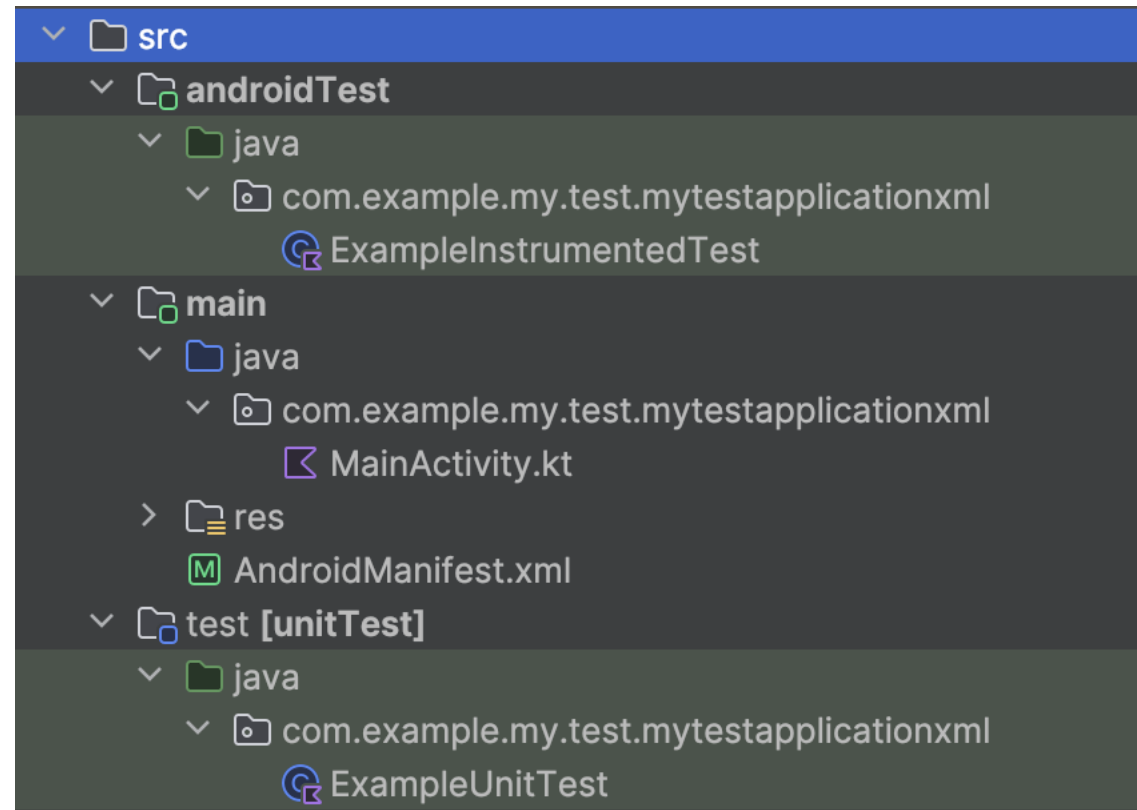
Empty Activity

Create a new empty activity with Jetpack Compose

Name	MyTestApplication
Package name	com.example.my.test.mytestapplication
Save location	/Users/mikhailshamburov/AndroidStudioProjects/MyTestApplication 
Minimum SDK	<input 7.0)"="" android="" nougat";="" type="text" value="API 24 ("/> 
 Your app will run on approximately 97,4% of devices. Help me choose	
Build configuration language 	Kotlin DSL (build.gradle.kts) [Recommended] 

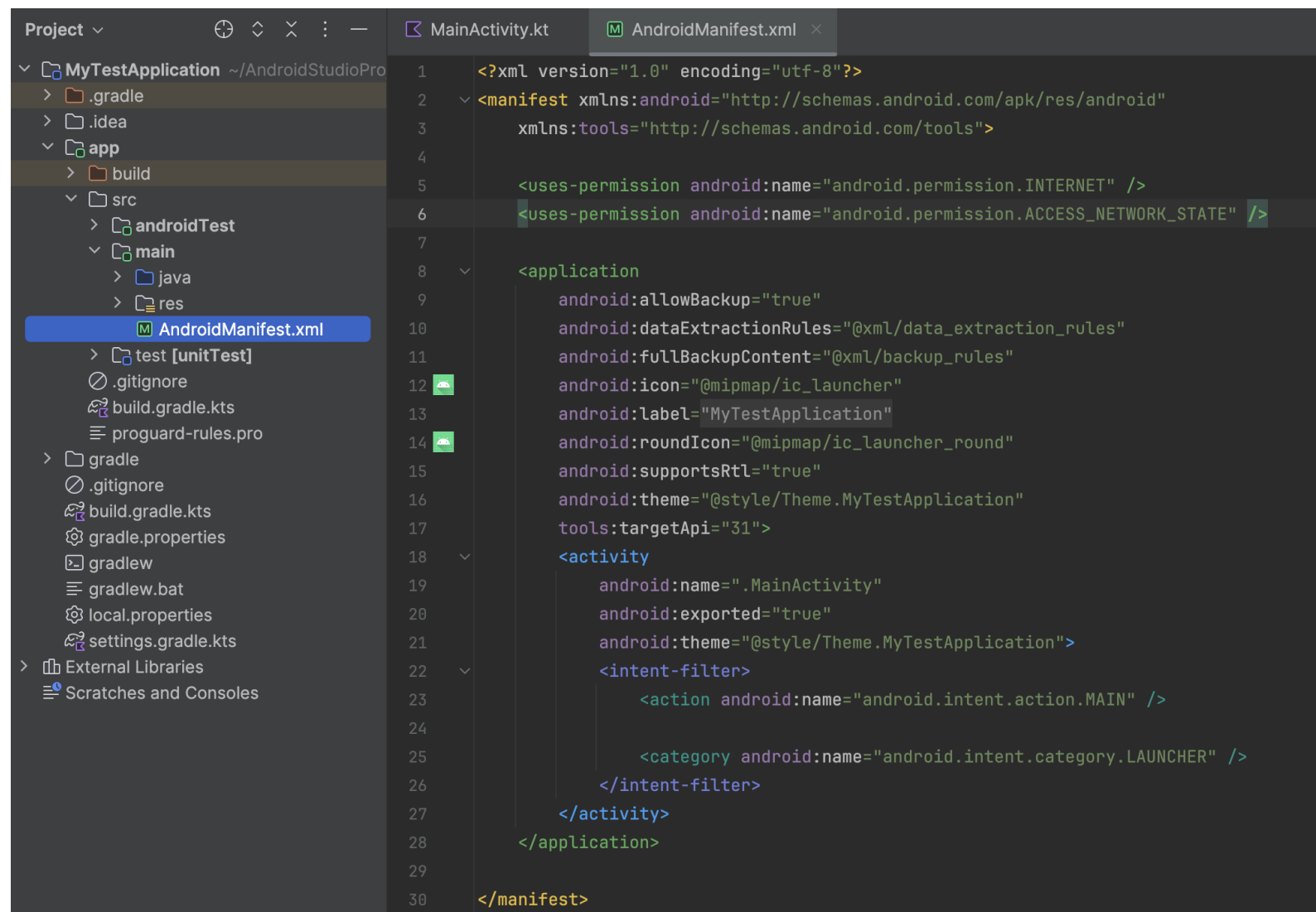
Android. Src

- **androidTest** - тесты с интеграцией android окружения;
- **test [unitTest]** - обычные unit-тесты без специфики android;
- **main/java** - файлы с исходным кодом приложения;
- **main/res** - файлы с различными ресурсами.



Android. Manifest

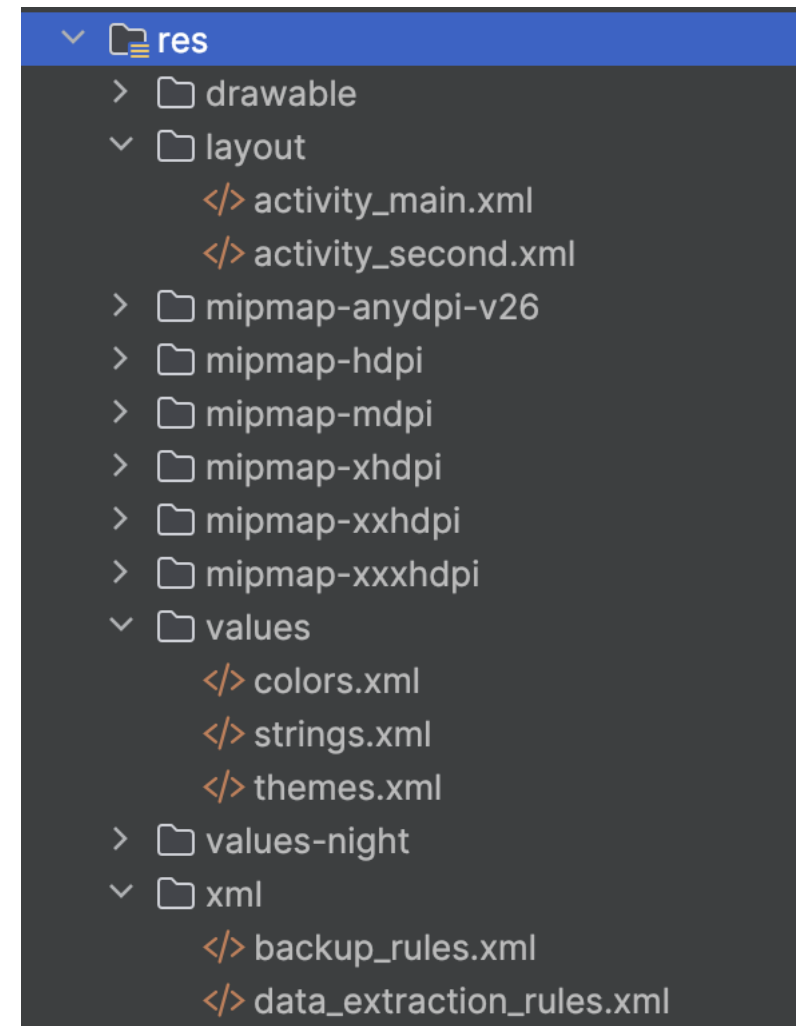
AndroidManifest - содержит информацию о приложении, его основных компонентах, разрешениях, настройках. Эту информацию использует система, для взаимодействия с приложением.



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <uses-permission android:name="android.permission.INTERNET" />
6     <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
7
8     <application
9         android:allowBackup="true"
10        android:dataExtractionRules="@xml/data_extraction_rules"
11        android:fullBackupContent="@xml/backup_rules"
12        android:icon="@mipmap/ic_launcher"
13        android:label="@string/app_name"
14        android:roundIcon="@mipmap/ic_launcher_round"
15        android:supportsRtl="true"
16        android:theme="@style/Theme.MyTestApplication"
17        tools:targetApi="31">
18
19        <activity
20            android:name=".MainActivity"
21            android:exported="true"
22            android:theme="@style/Theme.MyTestApplication">
23            <intent-filter>
24                <action android:name="android.intent.action.MAIN" />
25
26                <category android:name="android.intent.category.LAUNCHER" />
27            </intent-filter>
28        </activity>
29    </application>
30</manifest>
```

Android. Ресурсы

- **drawable** - для хранения jpeg, png, webp, bmaps, shapes, state lists, и т.д.
- **layout** - xml файлы, описывающие UI;
- **mipmap** - хранит иконку приложения для разных dpi;
- **values/colors** - цвета, используемые в приложении;
- **values/strings** - строковые ресурсы;
- **values/dimens** - константы размеров;
- **values/styles** - описание стилей xml элементов;
- **values/ids** - id константы;
- **values/themes** - xml файл с описанием темы приложения;
- **xml** - произвольные xml файлы;
- **font** - файлы шрифтов;
- **raw** - файлы произвольного формата: mp3, mp4 и т.д;



Android. app/build.gradle.kts

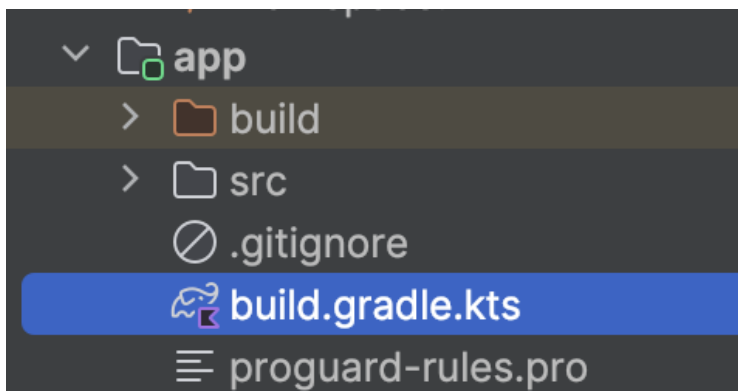
plugins - добавляет зависимости, которые дополнительно отрабатывают во время сборки проекта.

```
gradle dependencies Kotlin

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.jetbrains.kotlin.android)
}
```

plugins.android.application - плагин для сборки android приложения.

plugins.jetbrains.kotlin.android - плагин для поддержки сборки android проекта на Kotlin.



Android. app/build.gradle.kts

Содержит конфигурацию специфичную для android приложения:

gradle dependencies

Kotlin

```
android {  
    namespace = "com.example.my.test.mytestapplicationxml"  
    compileSdk = 34  
    defaultConfig {  
        applicationId = "com.example.my.test.mytestapplicationxml"  
        minSdk = 24  
        targetSdk = 34  
        versionCode = 1  
        versionName = «1.0»  
        //...  
    }  
    //...  
}
```

Android. app/build.gradle.kts

- **defaultConfig** - определяет общую конфигурацию для всех версий сборки.
- **applicationId** - уникальный идентификатор приложения в системе.
- **namespace** - используется как имя пакета при генерации файлов ресурсов R и BuildConfig. Рекомендуется, чтобы значение совпадало с **applicationId**.
- **minSdk** - минимальная поддерживаемая версия android api.
- **targetSdk** - версия, под которую ваше приложение адаптировано. Могут быть различия в поведении/требованиях к приложению на разных версиях android.
- **compileSdk** - версия инструментов для сборки приложения. Рекомендуется использовать одинаковую версию с **targetSdk**.
- **versionCode** - версия кода, не отображается для пользователя. Служит для отличия более старых и новых версий приложения.
- **versionName** - имя версии, отображается для пользователя.

Android. app/build.gradle.kts

gradle dependencies

Kotlin

```
android {  
    //...  
    buildTypes {  
        release {  
            isMinifyEnabled = false  
            proguardFiles(  
                getDefaultProguardFile(«proguard-android-optimize.txt»),  
                «proguard-rules.pro»  
            )  
        }  
    }  
    compileOptions {  
        sourceCompatibility = JavaVersion.VERSION_1_8  
        targetCompatibility = JavaVersion.VERSION_1_8  
    }  
    kotlinOptions {  
        jvmTarget = "1.8"  
    }  
}
```



Android. app/build.gradle.kts

- **buildType release** - переопределяет конфигурацию для релизной версии приложения;
- **compileOptions** - опции компиляции Java;
- **sourceCompatibility** - уровень версии Java для исходных файлов;
- **targetCompatibility** - целевая версия генерации байт-кода JVM;
- **kotlinOptions jvmTarget** - опции компиляции Kotlin. Целевая версия генерации байт-кода JVM;

Android. app/build.gradle.kts

Описывает зависимости на сторонние библиотеки/модули для сборки только конкретного модуля

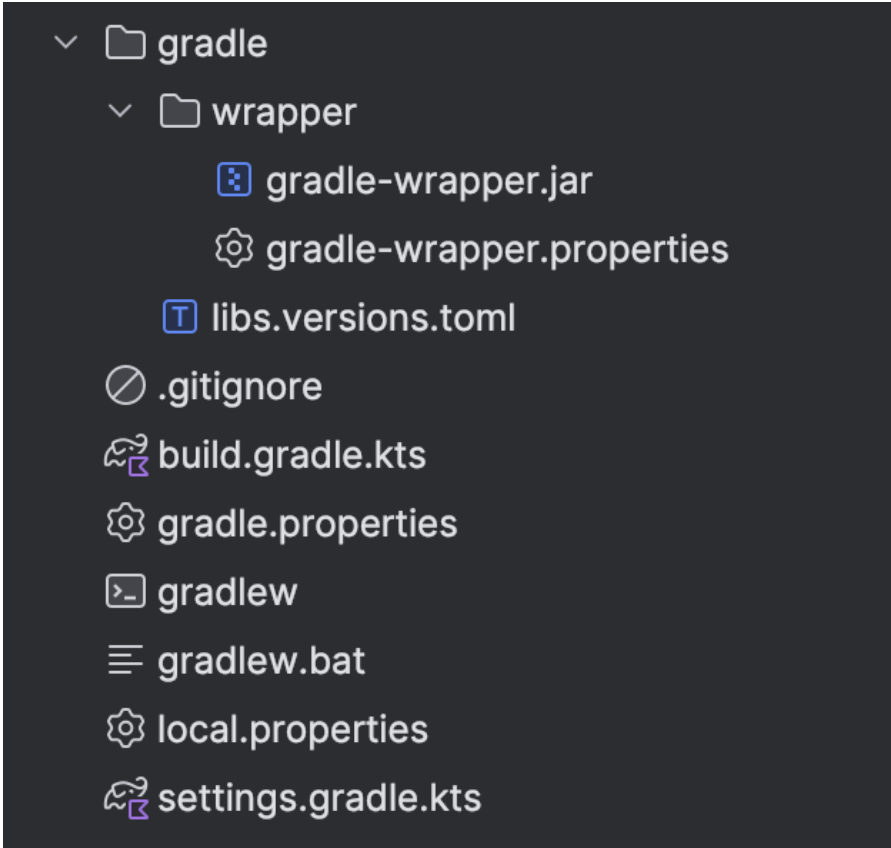
gradle dependencies

Kotlin

```
dependencies {  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
    implementation(libs.androidx.activity)  
    implementation(libs.androidx.constraintlayout)  
    testImplementation(libs.junit)  
    androidTestImplementation(libs.androidx.junit)  
    androidTestImplementation(libs.androidx.espresso.core)  
}
```

Верхнеуровневые gradle файлы

- **AGP** (Android Gradle Plugin) - плагин, расширяющий gradle, предоставляет специфическую функциональность для сборки android проектов;
- **gradle-wrapper.properties** - содержит информацию о версии используемой AGP;
- **libs.versions.toml** - каталог с описанием зависимостей используемых в проекте;
- **.gitignore** - файл с описанием правил исключения для git;
- **build.gradle.kts** - описывает настройки, применяемые для всех модулей проекта;
- **gradle.properties** - общие настройки gradle при сборке;
- **local.properties** - локальные настройки gradle при сборке.



```
gradle
├── wrapper
│   ├── gradle-wrapper.jar
│   └── gradle-wrapper.properties
├── libs.versions.toml
├── .gitignore
├── build.gradle.kts
├── gradle.properties
├── gradlew
├── gradlew.bat
├── local.properties
└── settings.gradle.kts
```

Settings.gradle.kts

Содержит информацию о репозиториях, для выкачивания зависимостей, и описывает, какие модули необходимо учитывать при сборке проекта.

settings.gradle.kts

Kotlin

```
pluginManagement {  
    repositories {  
        google {  
            content {  
                includeGroupByRegex("com\\.\\.android.*")  
                includeGroupByRegex("com\\.\\.google.*")  
                includeGroupByRegex("androidx.*")  
            }  
        }  
        mavenCentral()  
        gradlePluginPortal()  
    }  
}  
//...  
rootProject.name = "MyTestApplicationXml"  
include(":app")
```

Android. Основные компоненты

Фундаментальные строительные элементы, через которые система может взаимодействовать с Android приложением. Основные компоненты можно назвать точками входа в приложение.

При создании компонента, его необходимо добавить в AndroidManifest.

1. **Activity** - основная точка для осуществления взаимодействия с пользователем через UI.
2. **Broadcast receiver** - принимает сообщений от системы на различные события. Пример: будильники, пуши в приложение, состояние подключения к сети и т.д.
3. **Service** - используется при потребности в долго живущих задачах, работающих в фоне. Пример: воспроизведение музыки, загрузка больших файлов.
4. **ContentProvider** - управляет доступом к набору данных приложения. Через него, одно приложение может запросить какую-то информацию у другого приложения, например, список контактов.

Дефолтная реализация на Compose

MainActivity.kt

Kotlin

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyTestApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

Дефолтная реализация на View

MainActivity.kt

Kotlin

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

savedInstanceState - если активити было пересоздано, то данные сохраненные в bundle ранее (**onSaveInstanceState**), придут при вызове этого метода (а так же в **onRestoreInstanceState**). При первом открытии активити данные равны null.

setContentView - установить контент для отображения UI из файла ресурсов.

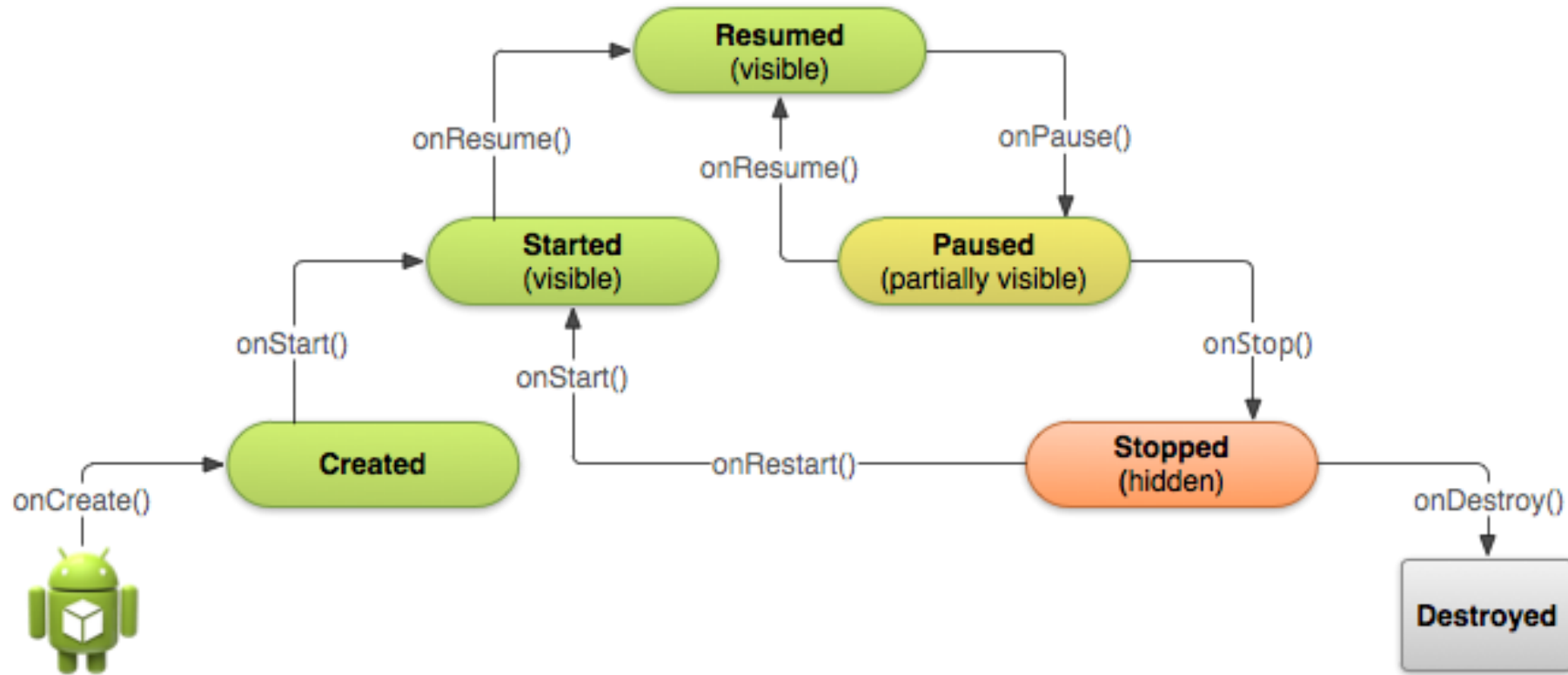
Дефолтная реализация на View

activity_main.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Android. Жизненный цикл Activity



Android. Context и Intent

Context - предоставляет информацию о приложении, системе, и методы для взаимодействия с ней.

Предоставляет доступ к ресурсам приложения, содержит методы на отправку интенгов, запуск других активити, сервисов, broadcast ресиверов и т.д.

Intent - описание какого-то действия к исполнению. Используется для описания какой компонент запустить, содержит параметры, которые будут переданы в этот компонент. Служит для коммуникации между приложениями и системой.

Может быть неявным, если указать, например:

```
Intent(Intent.ACTION_VIEW, Uri.parse(«https://some_addr.com»))
```

В данном случае откроется приложение, которое может обработать ссылку.

Отправка данных в другое Activity

MainActivity.kt

Kotlin

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        findViewById<Button>(R.id.send_rundom_number_button).setOnClickListener {  
            val context: Context = this@MainActivity  
            val intent = Intent(context, SecondActivity::class.java)  
                .putExtra("SomeNumber", (0..100).random())  
            context.startActivity(intent)  
        }  
    }  
}
```

Отправка данных в другое Activity

activity_main.xml

XML

```
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <Button
        android:id="@+id/send_rundom_number_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Send random number!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Получение данных от другого активити

SecondActivity.kt

Kotlin

```
class SecondActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_second)  
        val receivedNumber = intent.getIntExtra("SomeNumber", -1).toString()  
        findViewById<TextView>(R.id.description).text = receivedNumber  
    }  
}
```

Получение данных от другого активити

activity_second.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android=«http://schemas.android.com/apk/res/android»
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/description"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Получение данных от другого активити

